UNIVERSIDAD DE MÁLAGA

Departamento de Ingeniería de Sistemas y Automática

Sistemas de Tiempo Real

Dpto. de Ingeniería de Sistemas y Automática



Dr. D. Cipriano Galindo Andrades

UNIVERSIDAD DE MÁLAGA

Tema 7 – Sistemas Operativos para STR

7.1. Introducción

7.2. Aspectos de Tiempo Real en Sistemas Operativos

Administración de Tareas. Almacenamiento. Administración de Dispositivos.

7.3. Sistemas Operativos Específicos para Tiempo Real

Evolución Histórica. Estructuras Internas. Modos de Desarrollo. Facilidades provistas por un S.O.T.R. Multitarea. Repartidor. Problema de la Inversión de prioridades.

7.4. POSIX

7.5 FreeRTOS

7.6 Resumen

Departamento de Ingeniería de Sistemas y Automática

Tema 7 – Sistemas Operativos para STR

7.1. Introducción

Cuando existe (muchos microcontroladores no tienen), el sistema operativo es la base para cualquier aplicación.

Un S.O. moderno es tan complejo que no puede garantizar tiempo real duro en todas las circunstancias, pues debe garantizar a su vez una gran eficiencia en operaciones que entrarían en conflicto:

a) Administración de Tareas
b) Sistemas de Almacenamiento
c) Administración de Dispositivos y de E/S
d) Gestión de Usuarios

Para S.T.R. real real los S.O. de propósito general no suelen servir: existen S.O. especializados.

Los S.O. de T.R. normalmente sólo garantizan T.R. duro si se simplifica el conjunto de aplicaciones a ejecutar y si no se usan ciertos recursos (p.ej.: memoria virtual).

Tema 7 – Sistemas Operativos para STR

7.2. Aspectos de Tiempo Real en Sistemas Operativos

Administración de tareas.

Capacidad multitarea

No todos los sistemas operativos son multitarea, sobre todo los más pequeños.

No suelen ser deterministas ni pequeños los Tiempos de Cambio de Tarea (Task Switching Time): desde que una tarea pierde la CPU hasta que otra la toma.

Pueden tener un sistema multitarea costoso y lento (p.ej.: procesos UNIX)

Tema 7 – Sistemas Operativos para STR

7.2. Aspectos de Tiempo Real en Sistemas Operativos

Administración de tareas.

Planificación de tareas

Habitualmente permiten planificación basada en prioridades, pero en algunos casos puede ser no expulsiva (Windows 3.X).

Pueden tener escaso número de prioridades.

Pueden no tener solución para el problema de la inversión de prioridades (ver más adelante)

Comunicaciones y Sincronización de tareas

Habitualmente, no dan mecanismos deterministas (con timeouts, p.ej.) y no hay detección automática de algunos problemas (interbloqueos, inversión de prioridades,...)

Tema 7 – Sistemas Operativos para STR

7.2. Aspectos de Tiempo Real en Sistemas Operativos

Administración de tareas.

Seguridad/Protección

No todos los S.O.s protegen a las tareas del mal funcionamiento de otras tareas (no protegen sus espacios de memoria), especialmente los más pequeños.

Ejecución de tareas de núcleo

El núcleo del S.O. puede proveer rutinas no deterministas, no reentrantes, o que no se puedan expulsar de la CPU

Medida del tiempo

Suelen dar precisiones bajas (decenas de milisegundos), cuando deberían ser lo más precisas posible (aunque cuantificadas): microsegundos o mejor.

Tema 7 – Sistemas Operativos para STR

7.2. Aspectos de Tiempo Real en Sistemas Operativos

Sistemas de Almacenamiento

Determinismo en el tiempo

Habitualmente, no soportan determinismo en el tiempo, debido a:

Memoria virtual (debería poder inactivarse).

Fragmentación del disco (debería permitir preubicación de ficheros contiguos).

Administración de Dispositivos y de E/S

Puede que no soporten operaciones de E/S deterministas en el tiempo ni asíncronas.

Tema 7 – Sistemas Operativos para STR

7.2. Aspectos de Tiempo Real en Sistemas Operativos

Capacidad de Interrupción

Puede ser que el núcleo del S.O. no sea interrumpible / reentrante.

Las temporizaciones, basadas en interrupciones, pueden no ser exactas si no es determinista ni pequeño el Tiempo de Latencia de Interrupciones (Interrupt Latency Time): tiempo desde que la CPU recibe una señal de interrupción hasta que comienza la ejecución de la rutina de servicio de esa interrupción.

Accesos a Dispositivos Compartidos:

Las tareas suelen encolarse con estrategia FIFO para acceder a dispositivos compartidos, ignorándose sus prioridades y perdiendo así la planificación. El acceso a recursos debería estar ordenado primero por prioridad de la tarea, luego por su tiempo de llegada.

Tema 7 – Sistemas Operativos para STR

7.3. Sistemas Operativos específicos de Tiempo Real

Evolución Histórica

Los primeros sistemas operativos no fueron de tiempo real, sino de propósito general (CP/M, UNIX, MS-DOS, Windows, etc.).

Los primeros S.O.-T.R. estaban diseñados para aplicaciones muy concretas, normalmente en sistemas empotrados (automoción, control de vuelo, etc.).

Actualmente existen diversos sistemas operativos orientados al tiempo real, algunos útiles para sistemas empotrados (**QNX, LynxOS/LynuxOS, VxWorks**)

Se está dando cierta tendencia a estandarizar los S.O.T.R. con el objetivo de reducir los costes de portar un código de un S.O. a otro (p.ej.**: las normas POSIX**). El futuro de los S.O.T.R. está en los sistemas operativos de tiempo real distribuidos, orientados a componentes, y multiprocesador (ARTS, MARUTI, MARS, CHAOS, etc.).

Tema 7 – Sistemas Operativos para STR

7.3. Sistemas Operativos específicos de Tiempo Real.- Estructuras Internas

Monolíticos: el S.O. está compuesto de una única pieza de código, quizás dividida en módulos.



Ventajas:

-Incluye toda la funcionalidad necesaria en el núcleo.

-De fácil instalación y configuración.

Inconvenientes:

-Difícil de reducir para el caso de sistemas empotrados.

-Difícil de depurar. Las correcciones/parches de errores del S.O. pueden producir nuevos errores.

En Capas: el S.O. está dividido en capas superpuestas, con distintos niveles de abstracción.



Ventajas:

-Más configurable y aislado que los S.O. monolíticos.

Inconvenientes:

-Aún difícil de reducir y de depurar.

-No todos los servicios del S.O. se prestan a estar divididos en capas.

Departamento de Ingeniería de Sistemas y Automática

Tema 7 – Sistemas Operativos para STR

7.3. Sistemas Operativos específicos de Tiempo Real.- Estructuras Internas

Modulares o Cliente/Servidor: el S.O. está compuesto de diversos servidores: módulos aislados que proporcionan servicios a otros módulos llamados clientes. Normalmente existe una pequeña porción del S.O. que incluye la funcionalidad básica (multitarea, interrupciones, etc.), llamada *micronúcleo* o incluso *nanonúcleo.*



Ventajas:

-Muy fáciles de depurar.

-Muy fáciles de escalar a las necesidades de la aplicación (muy usados en sistemas empotrados).

-Permiten configurarse dinámicamente, cargando y descargando módulos.

Inconvenientes:

-Al estar más fragmentados, se incrementa el coste de las comunicaciones entre partes de la aplicación si éstas pasan por el S.O.

-Mayor complejidad de configuración (elección e instalación de los módulos adecuados).

Tema 7 – Sistemas Operativos para STR

7.3. Sistemas Operativos específicos de Tiempo Real.- Modos de desarrollo

Se consideran dos conceptos: **Host** (ordenador y sistema operativo donde se desarrolla la aplicación) y **Target** (ordenador y sistema operativo donde se ejecuta).



El Host es el mismo que el Target

La aplicación se desarrolla en el mismo ordenador en que se ejecuta.

El S.O. debe proveer un entorno de desarrollo adecuado, y al mismo tiempo, unas prestaciones de T.R. adecuadas.

Normalmente, las prestaciones de T.R. terminan perjudicadas.

Tema 7 – Sistemas Operativos para STR

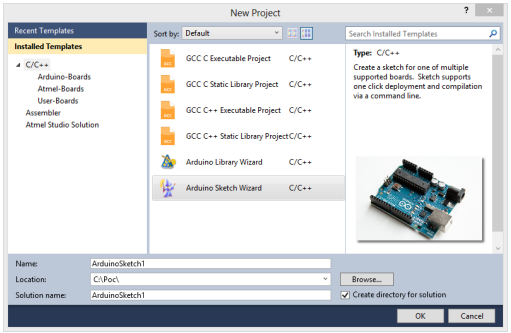
7.3. Sistemas Operativos específicos de Tiempo Real.- Modos de desarrollo

El Host es diferente que el Target



La aplicación se desarrolla en una máquina y luego se transfiere el código ejecutable a otra mediante red o conexión serie.

Permite utilizar máquinas de propósito general para el desarrollo, y máquinas especializadas para la ejecución.



Normalmente, las máquinas especializadas (Target) dan un tiempo real mejor que en el caso anterior.

Asimismo, el Host da un mejor entorno de desarrollo que en el caso anterior.

Se suele usar en sistemas empotrados → Target con pocos o ningún recurso de desarrollo (compiladores, editores, almacenamiento, ...).

Tema 7 – Sistemas Operativos para STR

7.3. Sistemas Operativos específicos de Tiempo Real.- Modos de desarrollo

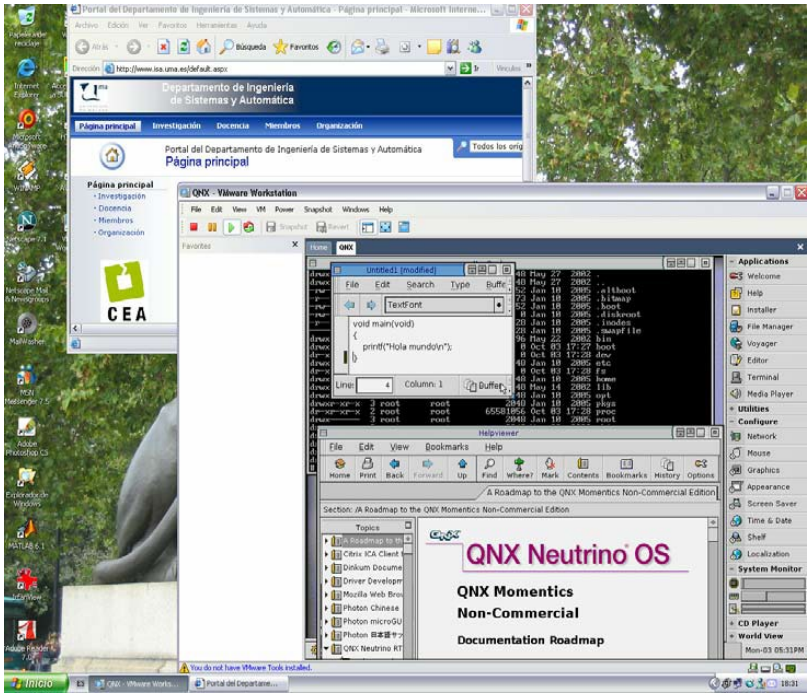
El Host y el Target comparten máquina pero no el mismo S.O.

En la misma máquina se ejecuta un S.O. de desarrollo (Host) y otro de ejecución (Target).

Se transfieren los ejecutables, por ejemplo, por medio de memoria compartida.

Si se ejecutan ambos S.O. simultáneamente, comparten hardware y recursos, lo que pone en peligro la eficiencia del Target.

Los sistemas con multiprocesadores ayudan en este caso.



Tema 7 – Sistemas Operativos para STR

7.3. Sistemas Operativos específicos de Tiempo Real.- Repartidor

Multitarea basada en *procesos* (pesados pero protegidos) y *threads* (ligeras pero no protegidas).

Asignación de la CPU basada en prioridades (producidas por alguna planificación): en todo momento, el proceso de mayor prioridad está usando la CPU si la necesita. Si varios de igual prioridad que la necesitan, hay varias estrategias de asignación:

A) FIFO con prioridades: todas las tareas de la misma prioridad se encolan con orden FIFO. Sólo cuando una de ellas libera voluntariamente la CPU, la siguiente la coge. Permite evitar accesos concurrentes a recursos entre tareas de la misma prioridad.

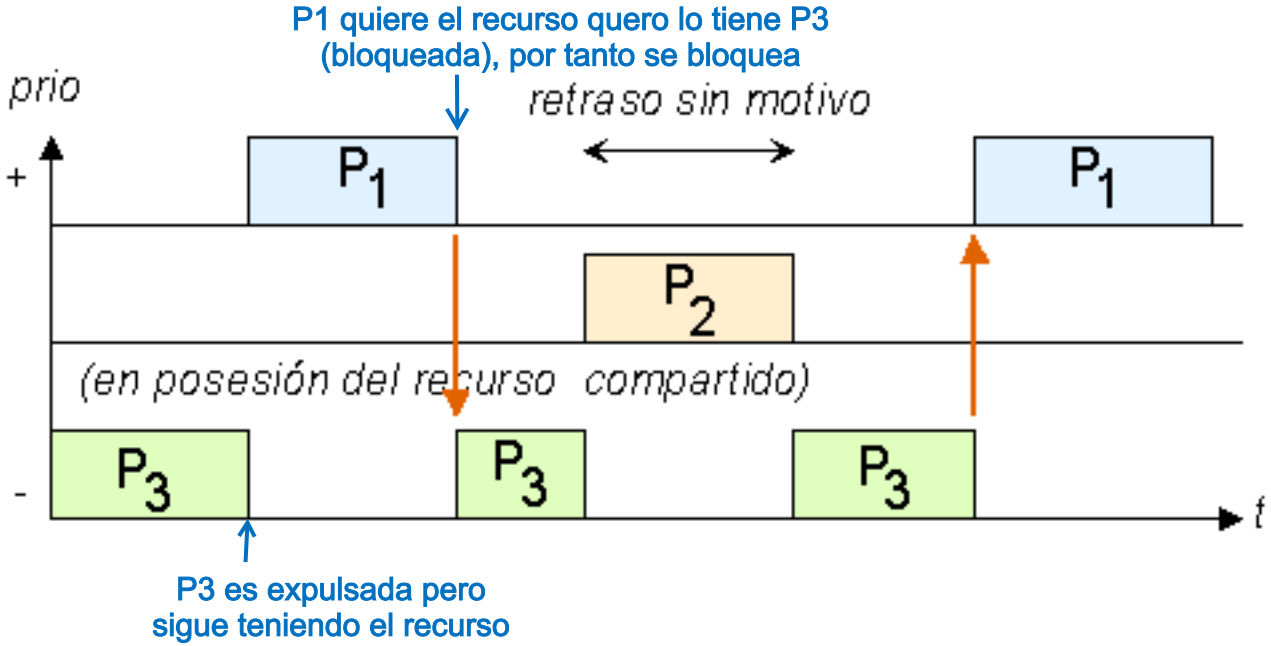
B) *Round Robin con prioridades*: igual que FIFO, pero hay un límite ("cuanto") de tiempo máximo que una tarea puede consumir. Al final del mismo, la CPU se le cede a la siguiente en orden FIFO y la que tenía la CPU pasa al final de la cola.
Se considera que da mejor rendimiento para tiempo real.

Tema 7 – Sistemas Operativos para STR

7.3. Sistemas Operativos específicos de Tiempo Real.- Repartidor

El problema de la Inversión de Prioridades

Sucede cuando hay recursos compartidos implicados: un proceso de alta prioridad acaba esperando indirectamente por otro de menor prioridad a causa de la espera por un recurso compartido. No se suele solucionar en S.O.s que no sean de tiempo real.

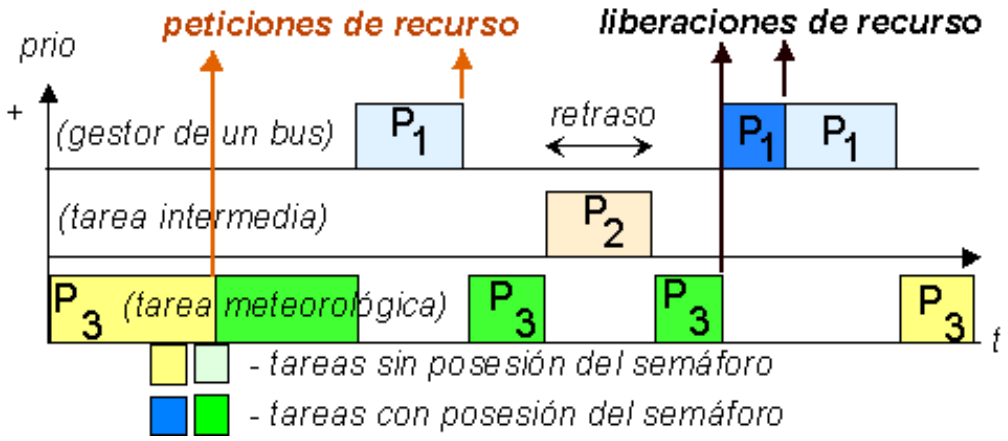


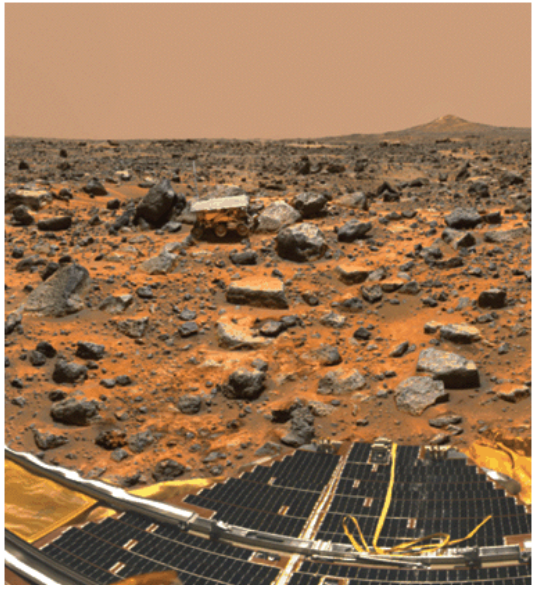
Tema 7 – Sistemas Operativos para STR

7.3. Sistemas Operativos específicos de Tiempo Real.- Repartidor

El problema de la Inversión de Prioridades

¡Sucedió en la Mars PathFinder en Julio de 1997! Los excesivos retrasos del gestor del bus (P1) causados por la tarea de prioridad intermedia (P2) fueron detectados por otra tarea superior (no mostrada), que reseteó la CPU por considerarlos peligrosos... una y otra vez.



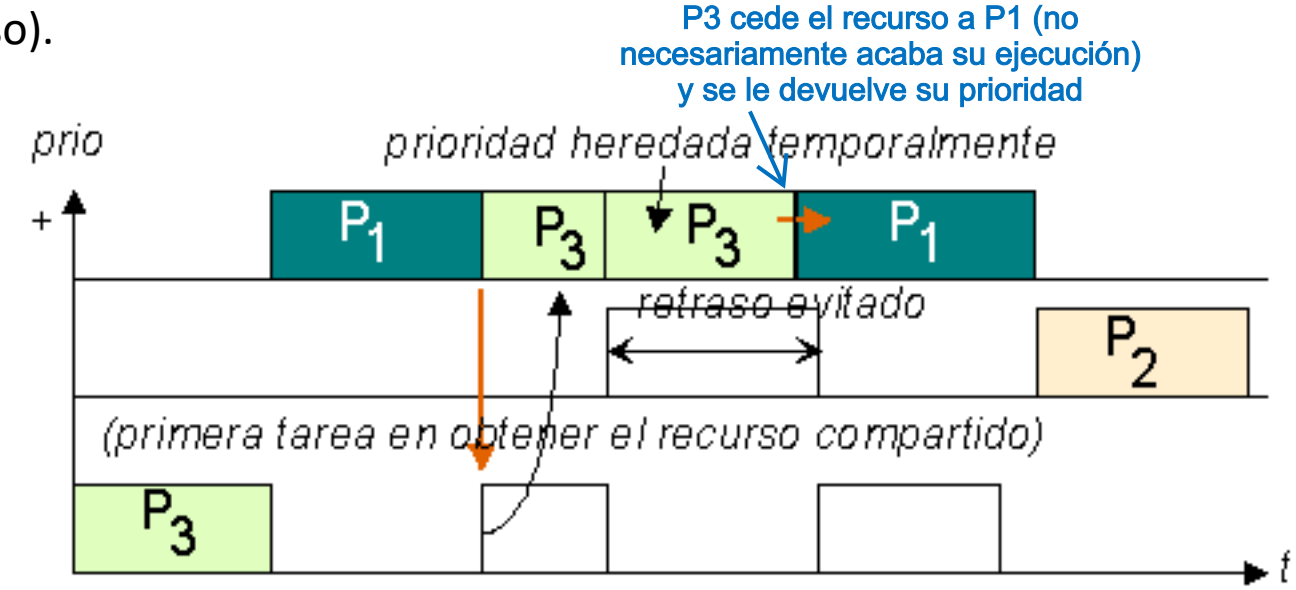


Tema 7 – Sistemas Operativos para STR

7.3. Sistemas Operativos específicos de Tiempo Real.- Repartidor

El problema de la Inversión de Prioridades

Primera Solución: herencia de prioridades (priority inheritance). La tarea que posee el recurso compartido hereda temporalmente la prioridad de la tarea de más prioridad de las que están pendientes de obtenerlo (en la gráfica la tarea P3 obtiene el recurso en el tiempo 0; las flechas son peticiones y cesiones del recurso).

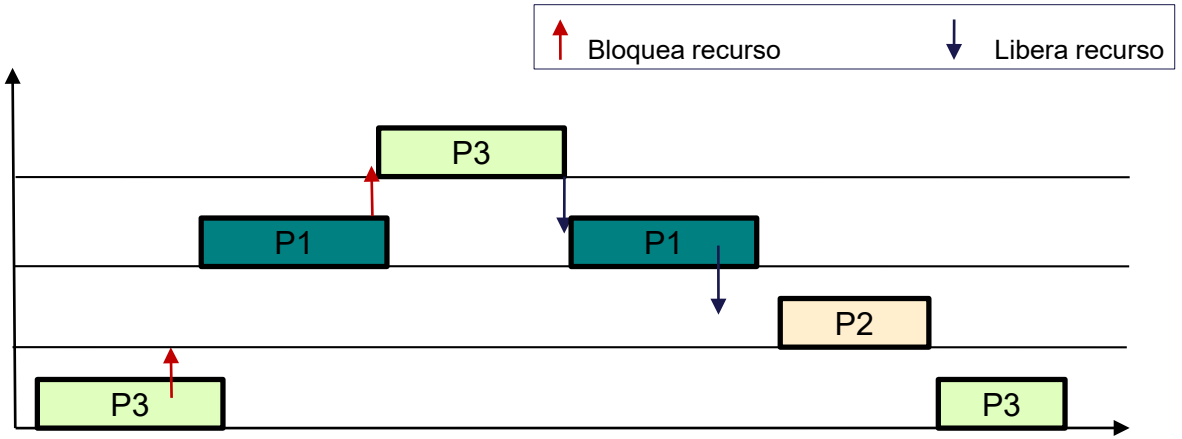


Tema 7 – Sistemas Operativos para STR

7.3. Sistemas Operativos específicos de Tiempo Real.- Repartidor

El problema de la Inversión de Prioridades

Segunda Solución: techos de prioridad (priority ceilings). A cada recurso compartido se le asocia una prioridad mayor en 1 que la prioridad mayor de todos sus clientes. Cuando una tarea se bloquea por un recurso, se le concede temporalmente esa prioridad a la tarea que lo tenía bloqueado (en la gráfica, P3 obtiene el recurso y la P1 debe esperar a que lo suelte). Tiene ventajas sobre la herencia: asegura que una tarea puede estar bloqueada sólo por un recurso y evita deadlocks indirectos.



Tema 7 – Sistemas Operativos para STR

7.4. POSIX: Portable Operating System Interface + UniX

- POSIX: conjunto de normas IEEE/ISO que definen la interfaz entre aplicaciones y el Sistema Operativo.
- Este estándar persigue portabilidad de las aplicaciones a nivel de código fuente.
- Está disponible para varios lenguajes como C, Ada, Fortran y otros.
- Diferentes versiones del standard: POSIX 1003.1, 1b, 1c…

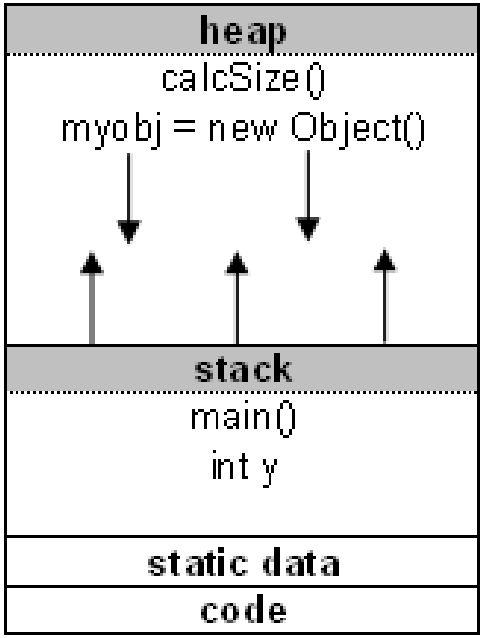
Departamento de Ingeniería de Sistemas y Automática

POSIX: Portable Operating System Interface + UniX

- POSIX 1003.1 define los servicios ofrecidos por Unix:
  - Gestión de procesos (Creación y destrucción, Sincronización, Temporización)
  - Gestión de archivos (Creación y borrado de archivos y directorios, protección, etc.)
- POSIX con extensión de TR (1003.1b):
  - Planificación de tareas, gestión de memoria, relojes y temporizaciones.
  - Funcionalidad para sincronización, semáforos, memoria compartida, etc.
- POSIX 1003.1c:
  - Inclusión de hilos (planificación, sincronización, etc.)

POSIX: Portable Operating System Interface + UniX

Un proceso/tarea es un programa en ejecución.

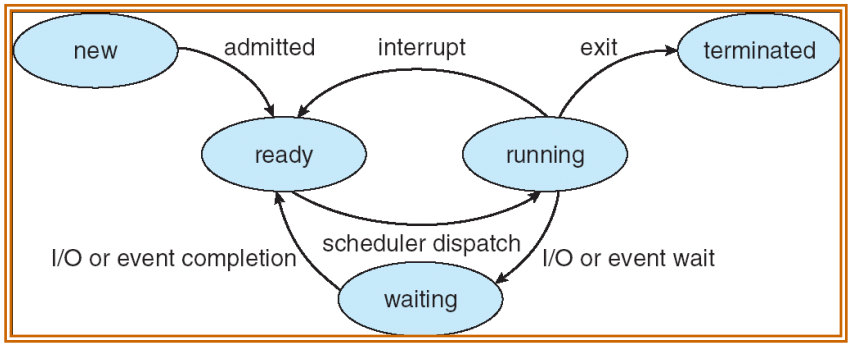


Es la unidad de ejecución más pequeña planificable e incluye:

- sección de código/ contador de programa
- sección de datos estática (ctes. y vble. globales)
- pila (variables) / heap (memoria dinámica)

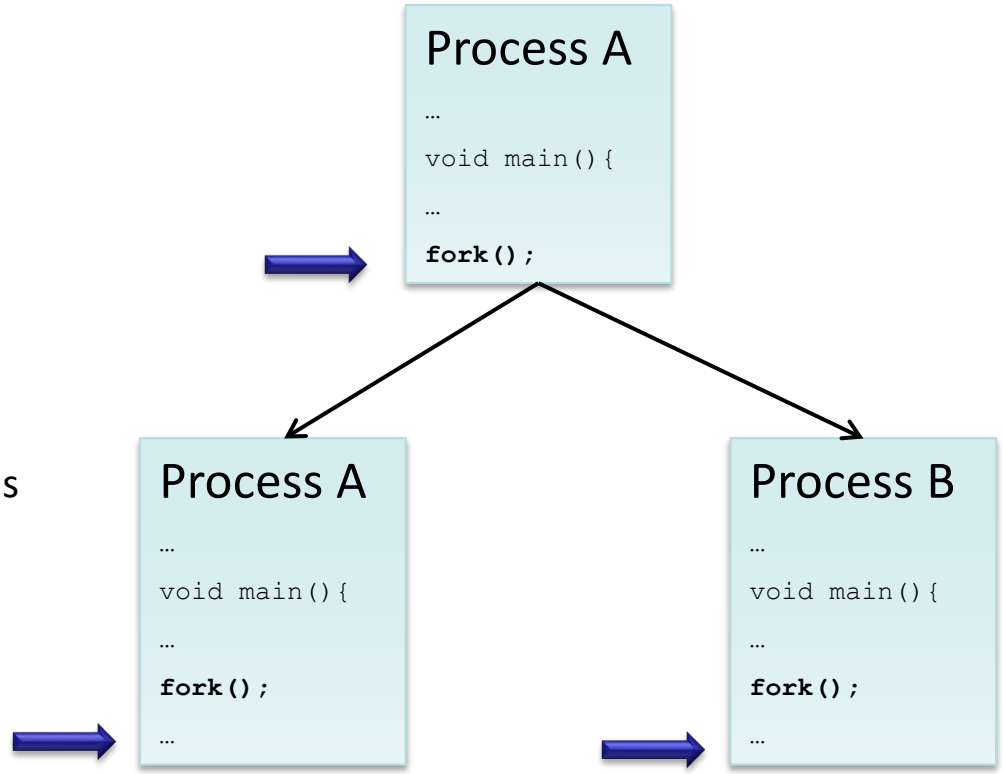
Conforme se ejecuta un proceso cambia su *estado*

- **nuevo:** El proceso se está creando
- **listo:** Está esperando que le asignen la CPU
- **en ejecución:** Se están ejecutando sus instrucciones
- **en espera:** Está esperando que ocurra algún evento (ej. E/S)
- **terminado:** Ha terminado su ejecución



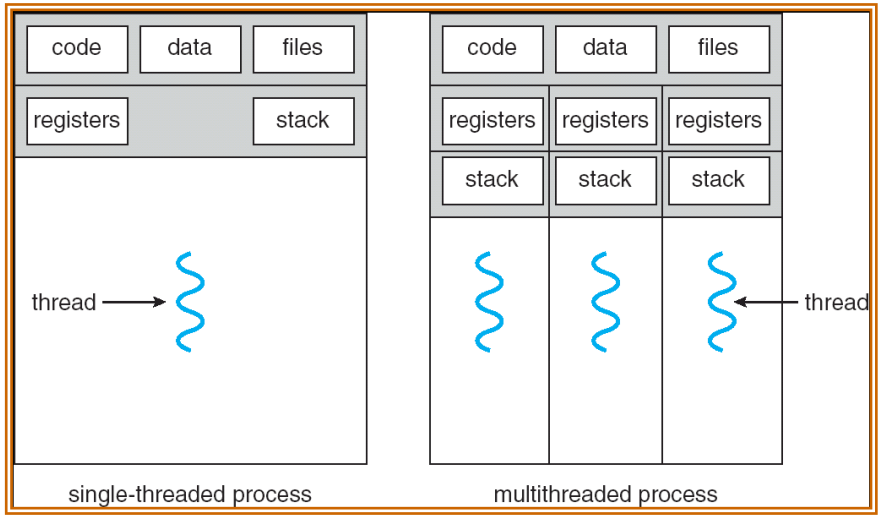
POSIX: Portable Operating System Interface + UniX

- Cada proceso mantiene:
  - Identificador de proceso o PID
  - Identificador de proceso padre o PPID
- La creación de procesos se realiza mediante una llamada a `fork()` que crea una **copia** (hijo) del proceso que la invoca
- El hijo hereda del padre:
  - Estado
  - Semáforos
  - Objetos de memoria
  - Política de planificación, etc.
- El hijo no hereda:
  - El PID
  - Alarmas y temporizadores
  - Operaciones de E/S asíncronas



POSIX: Portable Operating System Interface + UniX

- Una **hebra** es una unidad básica de utilización de la CPU consistente en un juego de registros y un espacio de pila. Es también conocido como **proceso ligero.** Comparte el código, los datos y los recursos con las hebras de su mismo proceso.
- Una tarea (o **proceso pesado**) puede estar formada por una o más hebras
- Una hebra sólo puede pertenecer a una tarea
- Se comparten recursos. La compartición de la memoria permite a las hebras comunicarse sin usar ningún mecanismo de comunicación del SO.
- No hay protección entre las hebras. Una hebra puede escribir en la pila de otra hebra del mismo proceso



POSIX: Portable Operating System Interface + UniX (Mecanismos de comunicación/sincronización)

- **Semáforo**: tipo de dato protegido que se utiliza para acceso exclusivo a recursos compartidos y para sincronización. Existen semáforos con y sin nombre. Semáforo está **disponible** si su valor es positivo y **no disponible** si vale 0.

- **Señal** es un mecanismo que permite avisar a los proceso de la llegada de un evento. Por ejemplo: división por cero, desbordamiento, expiración de alarmas, llegada de un mensaje, etc. Cada señal es identificada por un número. Cuando llega una señal, el proceso es interrumpido y se invoca a un manejador de señal.

SIGILL	Instrucción ilegal
SIGFPE	Operación aritmética errónea
SIGSEGV	Acceso a memoria erróneo
SIGINT	Señal de atención interactiva (Ctrl-c)
SIGUSR1	Reservada para el usuario
…	

POSIX: Portable Operating System Interface + UniX (Mecanismos de comunicación/sincronización)

- **Colas de mensajes**: información que se transfiere entre diferentes procesos o hilos mediante una cola. Cada mensaje lleva asociada una prioridad. Los mensajes se extraen de la cola de mensajes por orden de prioridad (no FIFO). Las colas de mensajes se identifican por medio de un nombre o un descriptor de cola de mensajes

- **Memoria compartida**: Zona de memoria reservada a la que pueden tener acceso varios procesos. Cada proceso que acceda puede escribir/leer y de esta forma varios procesos pueden compartir información y comunicarse.

- **Temporizaciones**: Dispone de funciones para obtener la "hora" del sistema y hacer esperas activas. La estructura para manera el tiempo es:

```
typedef struct timespec
{
        time_t   tv_sec;    /* seconds */
        long     tv_nsec;    /* nanoseconds */
} timespec_t;
```

- **Temporizadores**: Se pueden crear timers que se disparan pasado un tiempo programado para lanzar una señal, lanzar una hebra, etc.

7.5. FreeRTOS



FreeRTOS

- Pequeño sistema operativo que puede funcionar en casi cualquier microcontrolador.
  - Bajo consumo de memoria
  - Bajo consumo CPU
  - Sólo requiere de un temporizador (TIMER)
  - Diseñado para sistemas de tiempo real

- Ejecución concurrente de tareas
  - Mayor flexibilidad para el programador
  - Prioridad de ejecución
  - Semáforos
  - Colas de mensajes
  - …

FreeRTOS

- Configurable para que se adapte a diferentes plataformas.
- En el fichero FreeRTOSConfig.h, por ejemplo:

```
                                        …
#define configUSE_PREEMPTION                    1
#define configUSE_IDLE_HOOK                     1
#define configUSE_TICK_HOOK                     0
#define configCPU_CLOCK_HZ                      ( ( unsigned long ) 16000000)
#define configTICK_RATE_HZ                      ( ( portTickType ) 1000 )
#define configMAX_PRIORITIES                    ( ( unsigned portBASE_TYPE ) 4 )
#define configMINIMAL_STACK_SIZE    ( ( unsigned short ) 85 )
#define configTOTAL_HEAP_SIZE                   ( (size_t ) ( 1500 ) )
#define configMAX_TASK_NAME_LEN                 ( 8 )
#define configUSE_TRACE_FACILITY    0
                                        …
```

- Se ha modificado para utilizar el **timer 1** por el repartidor de tareas: cada tick de reloj (1 ms) comprueba qué tarea debe estar siendo ejecutada
- Las tareas se han modificado para que dejen una traza en la memoria RAM de la placa de desarrollo para trazar su ejecución.

FreeRTOS

- Creación de tareas:

```
#include "task.h"

xReturned = xTaskCreate(
                tskFunction,        /* Function that implements the task. */
                "NAME",             /* Text name for the task. */
                STACK_SIZE,         /* Stack size in words, not bytes. */
                ( void * ) 1,       /* Parameter passed into the task. */
                tskIDLE_PRIORITY,/* Priority of the task. */
                &xHandle );         /* handle to the task. */
```

tskFunction sería, por ejemplo, vTaskCode

- Función con el código de la tarea. Será el repartidor quien se encargue de llamarla, interrumpirla, y reanudarla.

```
void vTaskCode( void * pvParameters )
{

        for( ;; )
            {
                /* Task code goes here. */
            }
}
```

FreeRTOS

- Se inicia el repartidor llamando a la función:

```
vTaskStartScheduler();
```

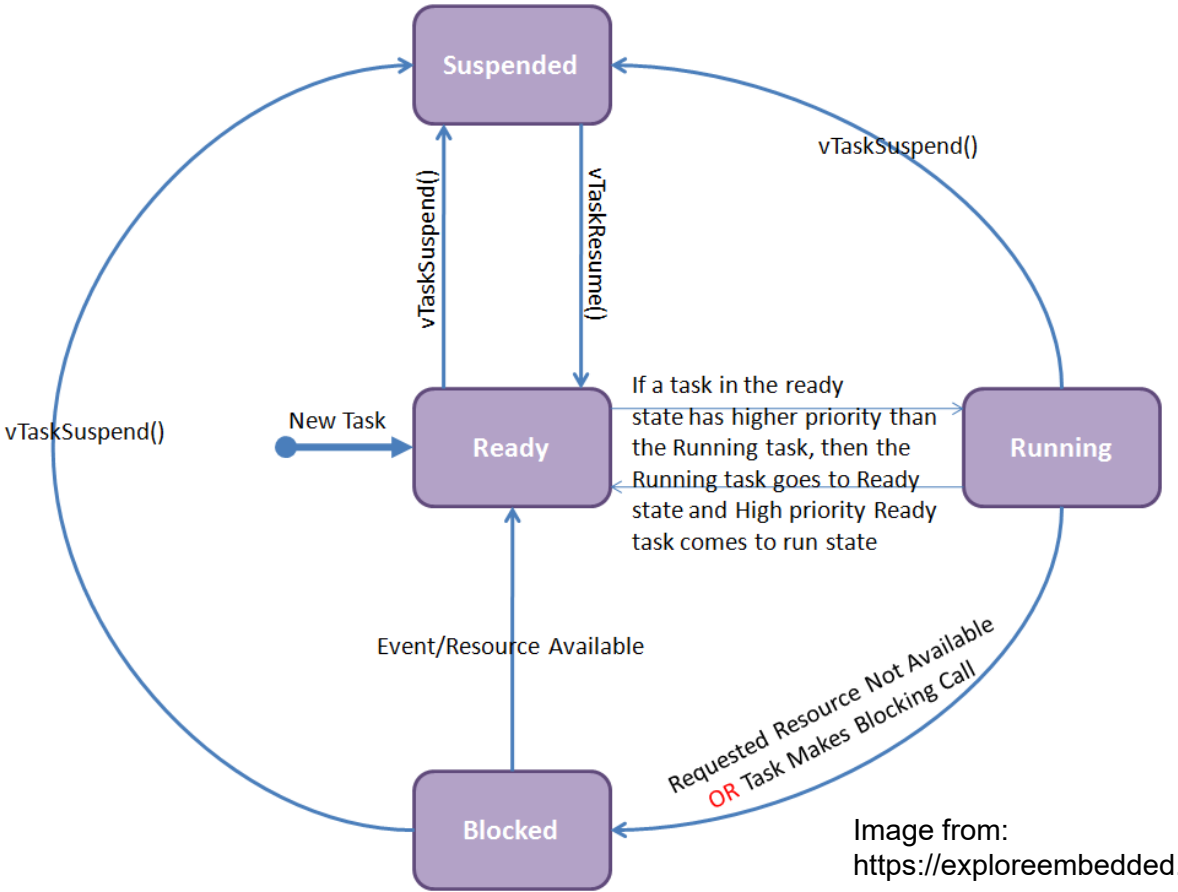
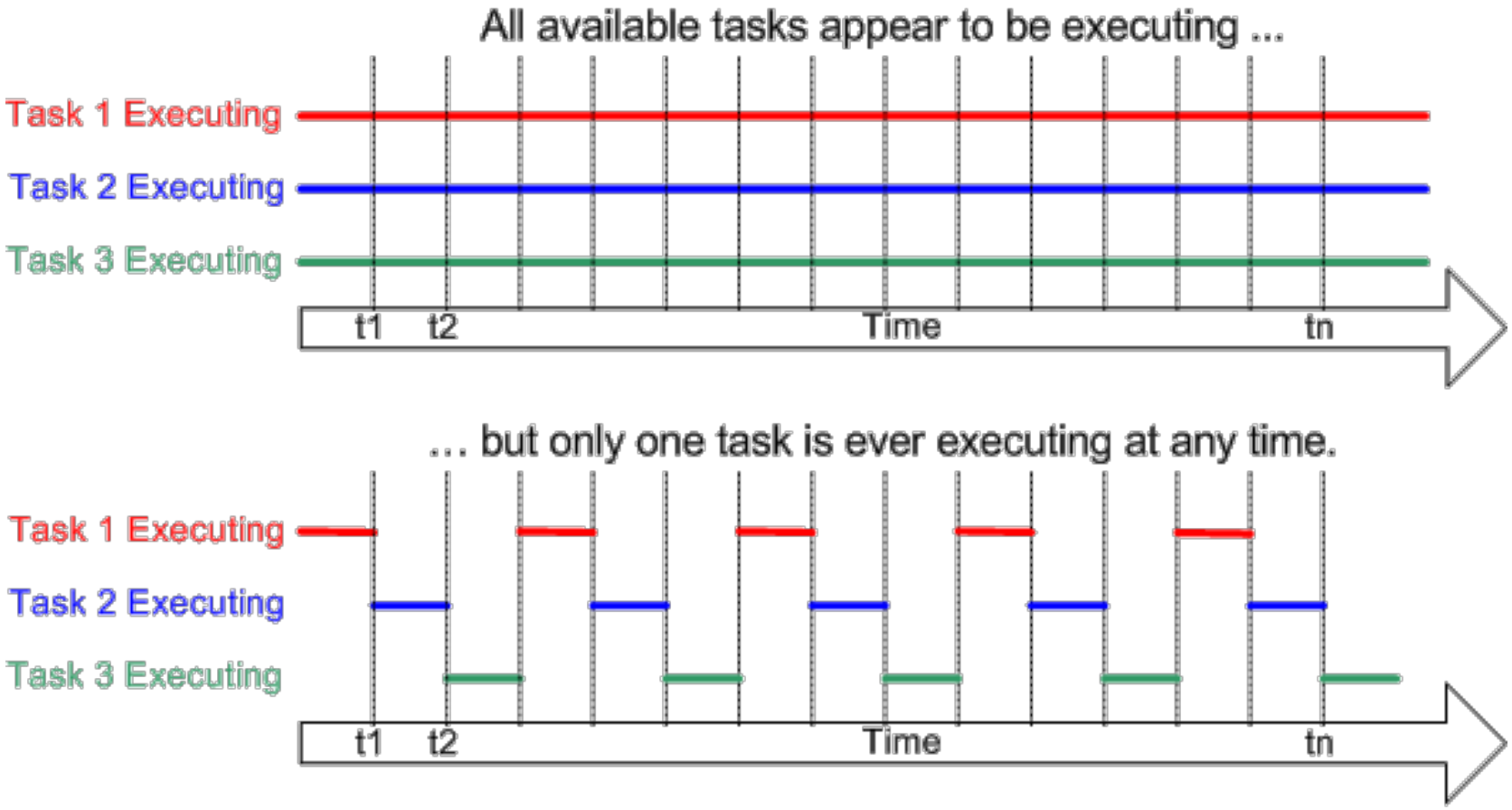


Image from:
https://exploreembedded.com/wiki/Task_Switching

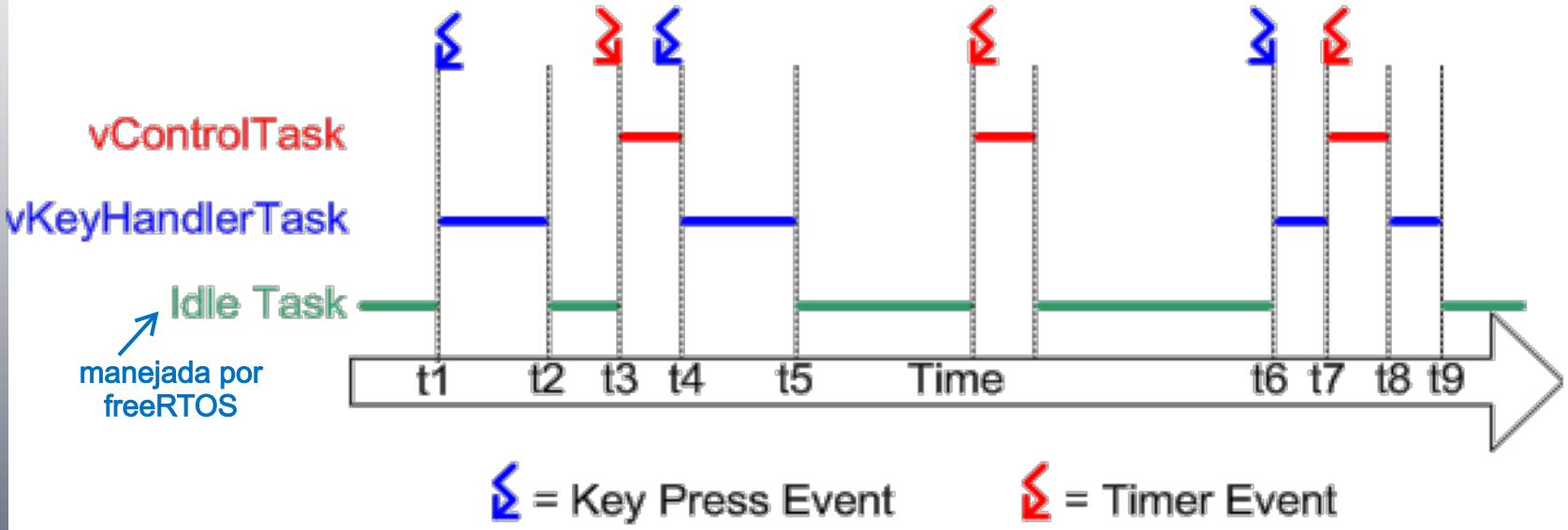
Tareas Concurrentes



Ticks Timer1

Departamento de Ingeniería de Sistemas y Automática

Tareas Concurrentes



Departamento de Ingeniería de Sistemas y Automática

Prioridades

- Las tareas con mayor prioridad tienen preferencia sobre las de menor
  - Pueden “quitar” CPU
  - Pero deben respetar secciones críticas de código (recursos bloqueantes).
- A igual prioridad, las tareas se turnan.

Como un RR de un cuanto de 1ms (tiempo entre cada observación de la CPU para ver a qué tarea le toca)

- Si no hay ningún tarea deseando ser ejecutada, se ejecuta “idle_task” por defecto.

Semáforo binario

- Actúa como una “llave” que permite acceso a un recurso. Si una tarea tiene la llave, los otros no pueden cogerla.
- Es responsabilidad del programador implementar un código que respete el semáforo.
- Si el programador no lo respeta, el código **puede** entrar en la sección critica ignorando las protecciones y corromper los datos de otras tareas.

```
#include “semphr.h”

SemaphoreHandle_t xSemaphoreCreateBinary( void );
```

Se crea en estado “vacío”. Así que hay que habilitarlo:

```
xSemaphoreGive(SemaphoreHandle_t xSemaphore)
```

Antes de tomarlo:

```
xSemaphoreTake( SemaphoreHandle_t xSemaphore,
                TickType_t xTicksToWait )
```

Mutex

Resuelve el problema de inversión de prioridades

Un mutex es un semáforo binario que **implementa herencia de prioridades**, por lo que está exento del problema de la inversión de prioridades.

```
SemaphoreHandle_t xSemaphoreCreateMutex( void )
```

Utilizan las mismas funciones que los semáforos binarios para las operaciones de "give" y "take"

Counting semaphores

Son semáforos que pueden tener más de una "llave", es decir, permiten ser accedidos más de una vez.

```
SemaphoreHandle_t xSemaphoreCreateCounting (
                UBaseType_t uxMaxCount,
                UBaseType_t uxInitialCount);
```

Cada vez que se realiza la operación de "give" se incrementa la cuenta en uno y cada vez que se hace "take" se decrementa. El semáforo está bloqueado cuando la cuenta llega a 0.

Tema 7 – Sistemas Operativos para STR

7.6. Resumen

- En este tema inicialmente se ha considerado los aspectos de tiempo real para la comunicación entre computadoras conectadas a una red.
- Adicionalmente, se han visto las características que poseen los Sistemas Operativos de propósito general que no son adecuadas para sistemas de tiempo real.
- Se ha descrito algunos S.O. de Tiempo Real, sus estructuras internas y modos de desarrollo
- Se ha visto el repartidor de tareas y un problema típico, el consistente en la inversión de prioridades, aportando dos soluciones al mismo.